

Faculty : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranab, Anup Keshri and Supratim Biswas
 Teaching Assistant : Swarna Aishwarya Twinkle (RS)

Lab Assignment 2 Objective : To get familiar with the use of Lex, the Lexical Analyzer generator tool. Write regular expressions manually for a few simple tokens, and write them in the format of Lex specifications. Generate the lexical analyzer (also named as scanner) through Lex. Experiment with the generated scanner and perform experiments to understand its working. Augment the scanner to add more functionality.

P1. Refresh the skills learnt in FLAT and use pen and paper for this problem. Consider an alphabet $\Sigma = \{a, b, c\}$ and consider the regular expression r , given by $R = R1 \mid R2$ over Σ ; where $R1 = b^*c^+$ and $R2 = a^*b$; $R1$ and $R2$ will be referred to as Token1 and Token2 respectively. Draw a minimal DFA, say D , for R .

Determine the behavior of D for the input string s , "aaabccc". D is supposed to be applied repeatedly till the entire string s is exhausted and the output will comprise of all the sub-strings of s that are recognized as tokens. For the input string "bbccbabc", D is expected to produce the report shown in the next column.	Token 1 found Token 2 found aToken 1 found Token 2 found Token 1 found
--	--

P2. You are given a lex script in the file "**regex1.l**" which has been uploaded in Babbage under /home/sb/..... Generate a scanner by performing the steps given in the handout, and give the executable a suitable name, such as "scan1". Three input files, with names "input1.txt", "input2.txt", "input3.txt", are given to you for experimentation. Run your generated scanner on all the 3 inputs, and submit the outputs generated along with your reasons.

P3. Consider the two modified versions of "regex1.l" given below.

regex2.l	regex3.l
%% b*c+ {printf("Token 1 found \n"); return 1;} a*b {printf("Token 2 found \n");return 2;} %% int main() { yylex(); }	%% b*c+ {printf("Token 1 found \n"); return 1;} a*b {printf("Token 2 found \n");return 2;} %% int main() { while(yyex()); }

Now repeat the steps of P2 to generate two scanners, from the lex scripts, and apply them to recognize the tokens in the input files provided. Submit all the 6 output files and report your findings and explain the cause of the behavior of the scanner over these inputs.

P4. Change the lex scripts of P2 / P3, so that the generated parser generates the outputs against the inputs, shown below.

Input	Output
bbccbabc	Token 1 : lexeme : bbcc Token 2 : lexeme : b Character a not recognized Token 1 : lexeme : c Token 2 : lexeme : ab Token 1 : lexeme : c

Input	Output
bbc c	Token 1 : lexeme : bbc
ba	Token 1 : lexeme : c
	Token 2 : lexeme : b
	Character a not recognized
cabc	Token 1 : lexeme : c
	Token 2 : lexeme : ab
	Token 1 : lexeme : c

P5. Change the lex scripts of P4 above, such that the generated parser generates the outputs against the input, shown below. Submit your lex script and generated output.

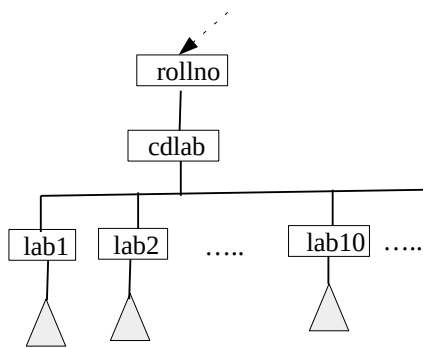
Input	Output
bbc c	Token 1 : bbc lexeme : 1
	Token 1 : c lexeme : 2
; ba \$	Character ; not recognized
	Token 2 : b lexeme : 1
x y z cabc #	Character a not recognized
	Character \$ not recognized
	Character x not recognized
	Character y not recognized
	Character z not recognized
	Token 1 : c lexeme : 3
	Token 2 : ab lexeme : 2
	Token 1 : c lexeme : 4
	Character # not recognized
	Total Lexemes found : 6
	Token 1 count = 4
	Token 2 count = 2

P6. In the remaining time, if any, experiment with the functioning of the scanners generated by you by creating your own input files and / or modifying the lex script to change the behavior to your liking.

1. Handout to illustrate the process of generating a scanner from a lex script.
2. Structure of a lex script
3. Lex scripts, regex1.l, regex2.l and regex3.l
4. input files, "input1.txt", "input2.txt" and "input3.txt"

End of Laboratory 2 Experiment

General Instructions : Directory Structure and Protocol for conduct of the laboratory sessions.



Create a separate subdirectory for each lab session within your subdirectory “cdlab”.

The answers to all the experiments conducted in a given lab session should reside in the tree under the directory for the lab.

Write a "submission" file by including all the programs that were written / run; outputs generated with reasons for their behavior. programs run, It will be retained by the instructors as your lab records and used for day to day evaluation.

It is advisable that you retain a copy to take care of your solutions and findings to avoid accidental deletion from the server.